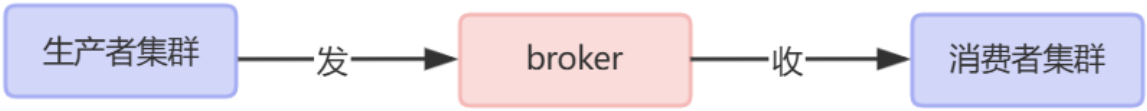
rabbitmq

基础

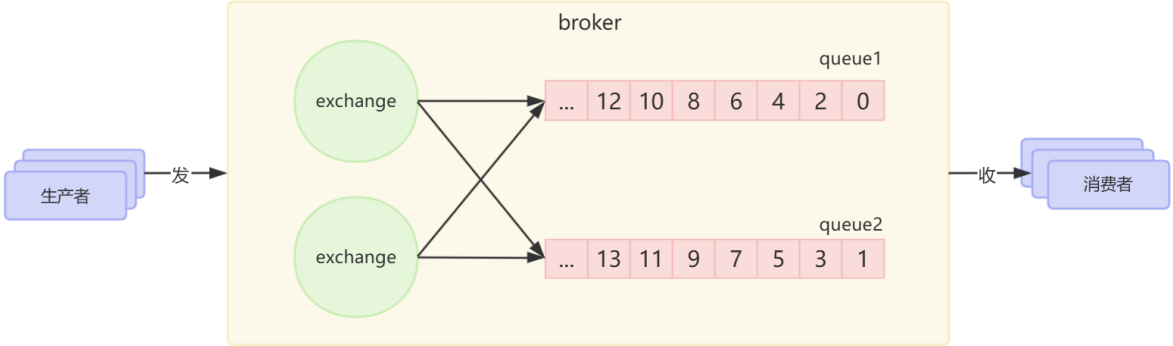
一个rabbitmq整体结构包括生产者、消费者、broker.



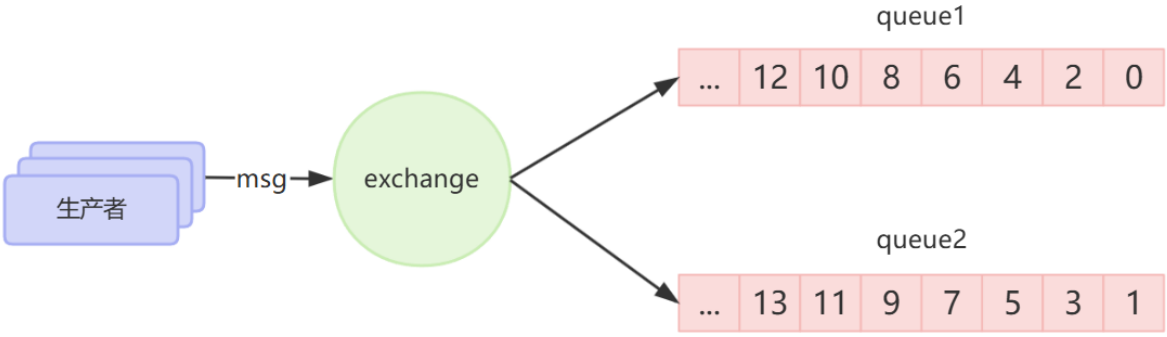
生产者就是发消息的, 消费者就是收消息的. broker可以理解为是rabbitmq本身, 负责接收生产者的消息, 并持久化到磁盘, 处理消费者的拉取请求.

kafka, rocketmq抽象出了topic的概念, 消息需要被发送到某个topic下, 虽然实际存储到了某个队列中, 但是队列的概念是对用户屏蔽的, 用户实际感知的是消息和topic.

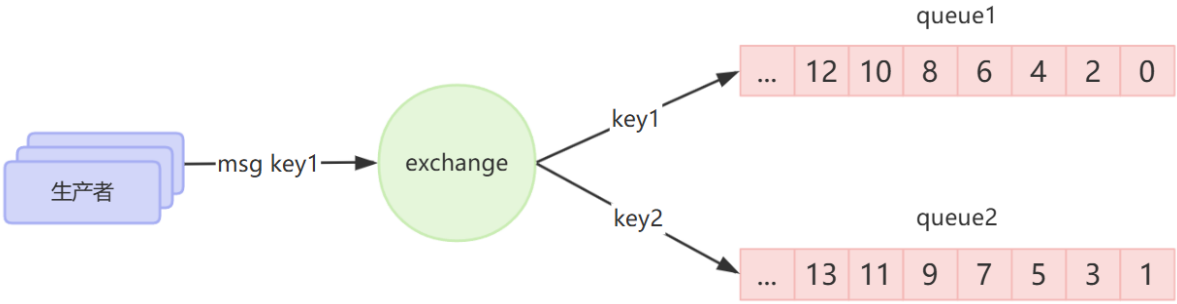
但是rabbitmq的设计完全不同, rabbitmq为了消息的灵活路由, 设计了交换机. 交换机通过一个binding key和队列绑定, 把消息转发给对应的队列, 然后消费者监听队列获取消息.



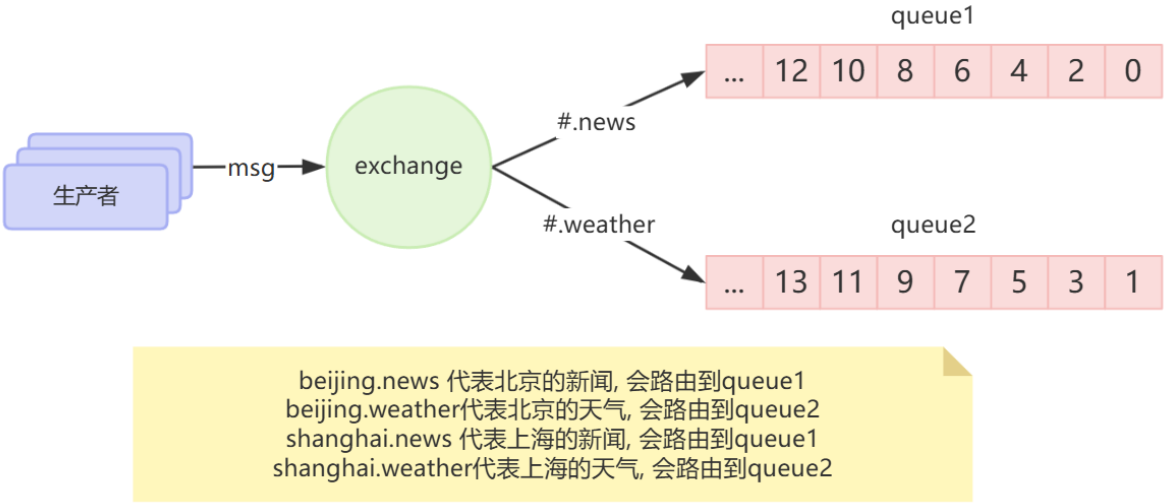
交换机有4种, 首先就是fanout交换机, 这个就是一种广播模式. fanout交换机会把消息路由到这个交换机绑定的所有队列中.



然后就是direct交换机, direct交换机会根据你发消息时的RoutingKey, 把消息路由到对应bindingkey对应的队列中.



第三种是topic交换机, topic交换机和direct交换机差不多, 但是在绑定队列的时候可以使用通配符匹配, 路由起来更加灵活.



第四种是header交换机, header交换机不依赖于bindingkey的匹配规则来路由消息, 它是通过消息中的 headers进行匹配的. 用起来麻烦, 性能也不好, 所以不推荐使用.

高可用

如果说rabbitmq只部署一个节点的话, 肯定会有单点故障问题, 所以怎么保证高可用呢?

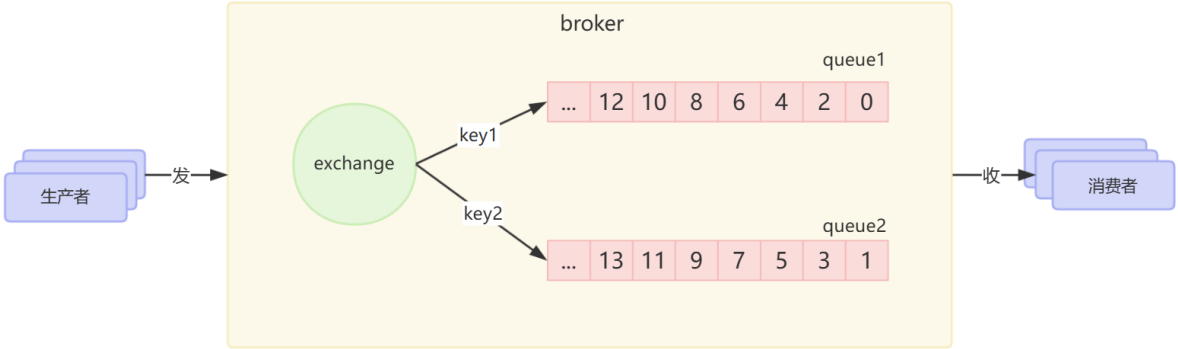
肯定得上集群啊. rabbitmq支持两种集群方式. 第一种叫普通集群, 普通集群模式下, 队列的元数据, 比如队列名称、属性, 会在所有集群节点间同步，但队列中的消息本身只存储在创建这个队列的节点上。其他节点仅持有指向该节点的元数据。这节省了存储空间，但是如果节点故障，该节点上的队列将无法使用，所以这种集群模式其实无法实现真正的高可用.

第二种集群方式叫镜像集群, 所谓镜像集群, 就是说集群中每个rabbitmq的节点都会有这个队列的完整镜像. 什么意思呢? 就是可以部署一个主节点, 然后部署几个镜像节点, 作为备份. 读写都是主节点, 然后主节点同步数据给镜像节点. 这和kafka的多副本机制类似, 相当于对队列做多个副本. 你写消息到某个队列的时候，它都会自动把消息同步到其他节点的队列上. 这其实就是一种多副本机制.

那么如果一个节点存不了全量数据怎么办呢? 很可惜rabbitmq原生不支持数据分片的能力, 它没办法把数据分布到不同的节点上. 但是官方提供了一个插件叫rabbit_sharding, 这个插件引入了一个分片交换机, 当你创建一个分片逻辑队列时，插件实际上会在后台创建多个物理分片队列, 物理的分片队列会分布在不同节点上. 然后通过分片策略将消息路由到不同节点的分片队列上.

如果你把我讲的这些视频都看下来, 你就会发现, 保证高可用, 无非就是两个东西, 一个叫数据冗余, 或者叫数据副本, 把一个数据复制多份放到不同的节点上, 这就是主从或者主备. 这就能够避免节点挂了带来的数据丢失问题. 实际案例就是mysql的读写分离, redis的主从架构, kafka的多副本机制, rocketmq的主从, rabbitmq的镜像集群. 另一个叫数据分片, 一个节点存不了全量数据, 那么就让集群的每个节点都只保存部分数据, 然后通过负载均衡算法, 让请求路由到对应的节点. 实际案例就是mysql的分库分表, redis的分片集群, kafka和rocketmq的集群模式, rabbitmq的sharding插件.

工作方式

rabbitmq的工作方式极其简单.

- 首先需要提前创建队列和交换机, 然后绑定.
- 生产者与 RabbitMQ Broker 建立连接, 将消息和routingkey发送到指定的交换机,
- 交换机根据自身的类型和绑定规则，将消息路由到队列里, 然后持久化保存到磁盘
- 消费者与 Broker 建立连接, 订阅指定的队列, 获取队列里的消息